
SAFE-M-ORP

Release 1

Mathis, François Méthivier

Jul 03, 2026

CONTENTS

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

1.1 Mesure module

Mesure.Graphe_T_V(*x*, *y*, *t_temps*)

summary Affiche deux graphiques en nuage de points des tensions mesurées (corrigées en mV) et de températures(°C) Crée une nouvelle fenêtre matplotlib intitulée ‘Graphique Voltage et Température et trace chaque valeur de y en fonction de son indice (axe temporel implicite).

Parameters

- **x** (*list[float]*) – Liste des températures mesurées à afficher (en °C).
- **y** (*list[float]*) – Liste des tensions corrigées à afficher (en V ou mV selon calibration).
- **t_temps** (*list[float]*) – Liste des temps écoulés depuis le début (en sec).

Mesure.connexion_port(*br=115200*, *portIN=""*)

summary Établit la connexion série avec l’Arduino. Si un port est fourni manuellement, tente de s’y connecter directement. Sinon, détecte automatiquement le port Arduino parmi les ports disponibles (Windows et Linux), en cherchant les identifiants caractéristiques dans le nom du port.

Parameters

- **br** (*int*) – Baudrate de la connexion série. Par défaut 115200.
- **portIN** (*str*) – Nom du port série à utiliser. Si vide, détection automatique.

Returns

- **portIN** : nom du port détecté ou fourni (ex. ‘COM6’, ‘/dev/ttyACM0’).
- **s** : objet `serial.Serial` connecté, ou la chaîne ‘error’ en cas d’échec.

Return type

`tuple[str, serial.Serial | str]`

Mesure.data(*s*, *N=None*)

summary Acquiert un nombre fixe de mesures depuis le port série, espacées d’un intervalle de temps connu (*time_inter*), et renvoie un temps écoulé en secondes pour chaque mesure (plutôt qu’un simple indice d’acquisition).

Parameters

- **s** (*serial.Serial*) – Objet `Serial` ouvert sur lequel lire les mesures.
- **N** (*int*, *optional*) – Nombre de mesures à effectuer. Si `None`, demandé à l’utilisateur.
- **time_inter** (*float*, *optional*) – Intervalle de temps (en sec) entre deux lectures. Par défaut 0.08 sec.

- **Returns** –

tuple[list[float], list[float], list[float]]:

- T : liste des températures mesurées (en °C).
- V : liste des tensions mesurées (en V).
- t_temps : liste des temps écoulés depuis le début (en sec).

Mesure.enregistrement_csv(*T, V_real, moy=None, sigma=None*)

summary Enregistre les données de température et de potentiel calibré dans un fichier CSV horodaté, accompagnées de la moyenne et de l'écart-type des tensions. Le fichier est sauvegardé dans le dossier Data/data_(T,V)/.

Parameters

- **T** (*list[float]*) – Liste des températures mesurées (en °C).
- **V_real** (*list[float]*) – Liste des potentiels calibrés (en mV).
- **moy** (*float, optional*) – Moyenne des tensions calibrées. Calculée automatiquement si non fournie.
- **sigma** (*float, optional*) – Écart-type des tensions calibrées. Calculé automatiquement si non fourni.

Returns

Message de confirmation d'enregistrement.

Return type

str

Mesure.enregistrement_pdf(*figure*)

summary Enregistre le graphique de température et de potentiel sous forme d'image PDF horodatée dans le dossier Data/data_figures/data_figures_mesures/.

Parameters

figure (*matplotlib.figure.Figure*) – Figure matplotlib à sauvegarder.

Returns

Message de confirmation d'enregistrement.

Return type

str

Mesure.graphe_csv(*chemin*)

summary Lit un fichier CSV contenant des données de température et de potentiel, puis trace un graphique de T et V en fonction du nombre de mesures. Le fichier CSV doit être horodaté et contenir deux colonnes : température (°C) et potentiel (mV).

Mesure.graphe_live(*CO, s*)

Mesure.informations_1er_ordre(*T, V, a*)

summary Calcule et affiche les statistiques de base d'une session de mesures : moyenne et écart-type des tensions calibrées et des températures mesurées. Affiche les résultats dans la console.

Parameters

- **T** (*list[float]*) – Liste des températures mesurées (en °C).
- **V** (*list[float]*) – Liste des tensions calibrées (en mV).

Mesure.t_attente(*nb=None*)

1.2 calibration module

`calibration.V_real_f(V, C0)`

summary Convertit une liste de tensions brutes en valeurs de potentiel corrigées (en mV), en appliquant la formule de conversion du capteur et l'offset de calibration C0. La conversion utilisée est : $V_{\text{corrigé}} = (2 - i) * 1000 + C0$.

Parameters

- **V** (*list[float]*) – Liste des tensions brutes mesurées par le capteur (en volts).
- **C0** (*float*) – Terme correctif issu de la calibration (en mV), peut être = 0 si calibration d'usine, ou une valeur calculée via `calibration_etalon()`.

Returns

Liste des potentiels corrigés (en mV), de même longueur que V.

Return type

`list[float]`

`calibration.calibration_2_etalons(s, E1=None, E2=None, V1=None, V2=None)`

summary Effectue une calibration à deux étalons par régression linéaire. Demande à l'utilisateur les potentiels des deux solutions étalons, acquiert les mesures sur chacune d'elles, puis calcule la droite de tendance par régression linéaire (`scipy.stats.linregress`) entre les tensions moyennes mesurées et les potentiels théoriques. Avertit si le R^2 est inférieur à 0.9.

Remarque : avec seulement 2 points, R^2 vaut toujours 1 par construction mathématique — l'avertissement $R^2 < 0.9$ ne peut donc jamais se déclencher.

Parameters

- **s** (*serial.Serial*) – Objet de connexion série avec l'Arduino.
- **E1** (*float, optional*) – Potentiel théorique de la solution étalon 1 (en mV).
- **E2** (*float, optional*) – Potentiel théorique de la solution étalon 2 (en mV).
- **V1** (*list[float], optional*) – Tensions brutes mesurées sur l'étalon 1.
- **V2** (*list[float], optional*) – Tensions brutes mesurées sur l'étalon 2.
- **V_real1** – tension convertit en mV
- **V_real2** – tension convertit en mV

E1 et E2 sont demandé à l'utilisateur si non fournis, V1 et V2 sont acquises via `mes.data()` si non fournies V_real1 et V_real2 sont acquise via V1 et V2

Returns

(C0, V1_moy, V2_moy, tendance, r_squared, E1, E2)

- **C0** (*float*): Intercept de la droite, utilisé comme offset de calibration (en mV).
- **V1_moy** (*float*): Tension moyenne mesurée sur l'étalon 1 (en mV).
- **V2_moy** (*float*): Tension moyenne mesurée sur l'étalon 2 (en mV).
- **tendance** (*np.poly1d*): Droite de calibration sous forme de polynôme.
- **r_squared** (*float*): Coefficient de détermination R^2 de la régression.
- **E1** (*float*): Potentiel théorique de l'étalon 1 (en mV).
- **E2** (*float*): Potentiel théorique de l'étalon 2 (en mV).

Return type

tuple

`calibration.calibration_etalon(V)`

summary Calcule le terme correctif C0 par calibration à un étalon. Demande à l'utilisateur la valeur du potentiel de la solution étalon, puis calcule pour chaque tension mesurée l'écart entre la valeur attendue et la valeur convertie. C0 est la moyenne de ces écarts, utilisée ensuite pour corriger les mesures réelles.

Parameters

V (*list[float]*) – Liste des tensions brutes mesurées sur la solution étalon (en volts).

Returns

C0, terme correctif moyen (offset en mV) à appliquer aux mesures futures.

Return type

float

`calibration.calibration_usine()`**_summary_**

Applique la formule de calibration d'usine à une liste de tensions mesurées. Pour chaque valeur de V ('brut'), on convertit : $E = (2 - i) * 1000$ (qui sera calculé par V_real), mais retourne systématiquement C0 = 0 (aucun offset correctif n'est appliqué). Cette fonction sert de référence sans correction : elle suppose que le capteur est déjà calibré en sortie d'usine.

Remarque : on ne savait pas si on devait définir une fonction (cacalibration_usine) ou juste mettre C0 = 0 pour l'option 1 du menu

Args:

none

Returns:

int: C0 = 0, terme correctif nul (calibration d'usine sans offset).

`calibration.enregistrement_cal(C0, tendance=None, nb_etalons=None)`

summary Enregistre les paramètres de calibration dans un fichier CSV horodaté. Selon le nombre d'étalons, le fichier contient soit uniquement C0 (calibration à 1 étalon ou d'usine), soit C0, la pente et l'intercept de la droite de tendance (calibration à 2 étalons). Le fichier est sauvegardé dans le dossier Data/data_calibration/.

Parameters

- **C0** (*float*) – Terme correctif issu de la calibration (en mV).
- **tendance** (*np.poly1d, optional*) – Objet polynôme contenant la pente et l'intercept de la droite d'étalonnage. None si calibration à 1 étalon.
- **nb_etalons** (*int, optional*) – Nombre d'étalons utilisés (1 ou 2). None si calibration d'usine.

Returns

None

`calibration.enregistrement_cal2_pdf(fig)`

summary Enregistre le graphique de calibration à 2 étalons sous forme d'image PDF horodatée dans le dossier Data/data_figures/data_figures_calibration/.

Parameters

fig (*matplotlib.figure.Figure*) – Figure matplotlib à sauvegarder.

Returns

Message de confirmation d'enregistrement.

Return type

str

calibration.**graphe_cal2**(*tendance*, *a*, *C0*, *E1*, *E2*, *V1_real*, *V2_real*)

__summary__ Affiche le graphique de la droite de calibration à 2 étalons. Trace la droite de calibration $E = a * V_real + C0$ ainsi que les deux points étalons, avec l'équation de la droite en légende.

Parameters

- **tendance** (*np.poly1d*) – Droite de calibration sous forme de polynôme, construite à partir de *a* et *C0* via `np.poly1d([a, C0])`.
- **a** (*float*) – Pente de la droite de calibration (sans unité, idéalement 1).
- **C0** (*float*) – Intercept de la droite de calibration (en mV).
- **E1** (*float*) – Potentiel théorique de la solution étalon 1 (en mV).
- **E2** (*float*) – Potentiel théorique de la solution étalon 2 (en mV).
- **V1_real** (*float*) – Tension mesurée convertie de l'étalon 1 (en mV), obtenue via $(2 - V1_moy) * 1000$.
- **V2_real** (*float*) – Tension mesurée convertie de l'étalon 2 (en mV), obtenue via $(2 - V2_moy) * 1000$.

Returns

Figure matplotlib du graphique de calibration.

Return type

matplotlib.figure.Figure

1.3 consigne module

1.4 test module

test.**graphe_cal2**(*tendance*, *a*, *C0*, *E1*, *E2*, *V1_real*, *V2_real*)

__summary__ Affiche le graphique de la droite de calibration à 2 étalons. Trace la droite de calibration $E = a * V_real + C0$ ainsi que les deux points étalons, avec l'équation de la droite en légende.

Parameters

- **tendance** (*np.poly1d*) – Droite de calibration sous forme de polynôme, construite à partir de *a* et *C0* via `np.poly1d([a, C0])`.
- **a** (*float*) – Pente de la droite de calibration (sans unité, idéalement 1).
- **C0** (*float*) – Intercept de la droite de calibration (en mV).
- **E1** (*float*) – Potentiel théorique de la solution étalon 1 (en mV).
- **E2** (*float*) – Potentiel théorique de la solution étalon 2 (en mV).
- **V1_real** (*float*) – Tension mesurée convertie de l'étalon 1 (en mV), obtenue via $(2 - V1_moy) * 1000$.
- **V2_real** (*float*) – Tension mesurée convertie de l'étalon 2 (en mV), obtenue via $(2 - V2_moy) * 1000$.

Returns

Figure matplotlib du graphique de calibration.

Return type

matplotlib.figure.Figure

PYTHON MODULE INDEX

C

calibration, ??

m

Mesure, ??

t

test, ??